

The GOATS Loop, Quick Reference

Frame the problem before you build the agent.

GOATS is five steps, run before you build, run again when something breaks. This is the short version.

The five steps

Step	Question	Output
1. Goal	What decision are we supporting? How will we measure success?	Business metric and success definition
2. Operating Assumptions	What would have to be true? What's the atomic unit?	Testable hypothesis list
3. Alternatives	What approaches could work?	2-4 candidate approaches
4. Trade-offs	How do we choose?	Justified selection
5. Signals	How will we know if we're wrong?	Stop criteria and leading indicators

Step 1: Goal

Step 1 is done when you can fill in the Goal Statement:

We help **[owner]** make **[decision]** better. We'll know it's working when **[metric]** moves. We are explicitly not building **[non-goals]**. If the system were perfect tomorrow, it would still fail unless **[Oracle Test answer]** is handled. It's worth building because **[one-line ROI]**.

Blank	Red flag
Decision	"Build a chatbot" (a thing the system produces, no decision named)
Owner	"The business" (nobody whose budget moves if it works)
Metric	"Improve customer experience" (too vague to move)
Non-goals	None written down
ROI	Value counted at pilot scale, with no cost of errors

If a blank stays empty, don't push forward. The empty blank is the finding.

The Oracle Test. *If this agent was 100 percent accurate right now, what would still cause it to fail?* If the answer is short, you have a model problem. If the answer is long, you have a framing problem dressed as one.

The metric is the frame. Two teams with the same model and the same data will build different systems depending on which metric they pick.

Step 2: Operating Assumptions

The atomic unit. *What does the system decide about, one at a time?*

Name three things separately. The unit is what the system handles one at a time. The stages are what it does to that unit. The outcome is what you grade one unit on. The common mistake is writing the stages on the slide and calling them the unit.

What the slide says	The unit	The stages inside it	The outcome you grade
"Handle customer issues"	One support ticket at triage	Classify, retrieve the policy, draft	A draft a human can approve without edits
"Review pull requests"	One potential issue in one file	Locate, explain, cite the evidence	A flag the reviewer acts on
"Research this topic"	One candidate source	Find, extract the claim, assess credibility	A claim with a credibility call a human agrees with

Three tests for an atomic unit: hold one in your hand, count them, get ground truth for one.

The math. Per-step reliability compounds.

Steps	Per-step	End to end
3	85%	61%
10	85%	20%
3	95%	86%
10	95%	60%

Lifting every step from 85 to 95 percent buys about what cutting ten steps to three gives you. Length is the cheaper lever.

The assumptions. List what would have to be true. Write each one so you can test it in a day, and note the cost if it's wrong. High cost and thin evidence means test it first.

Step 3: Alternatives

Two questions, in order. *Do we need AI at all? If yes, does it need to be autonomous?*

The automation spectrum:

Level	What you decide	What the LLM decides
Rules	Everything	Nothing
Constrained output	Structure and constraints	Content within them
Guided agent	Goal and guardrails	How to achieve it
Full agent	Only the goal	Everything else

The more you know about task structure, the less the LLM should decide.

Knowledge and skills. For each step the agent performs:

Type	Architecture
Established knowledge (the model knows it)	Prompt it. Minimal scaffolding.
Retrievable knowledge (info exists, model doesn't have it)	RAG, tool use, context injection
Teachable skills (model doesn't know the process)	Few-shot, fine-tune, structured workflow

Put two to four candidates on the slate that are different systems, not variations on the one you already wanted. One simpler. One more autonomous. One non-AI. Document the rejected ones, because constraints change.

Chapter 2 of the book, free at aiframer.dev/book, walks the alternatives step on a full project.

Step 4: Trade-offs

Score each alternative across six dimensions.

Dimension	The question
Cost	What does it cost to run?
Latency	How fast does it need to respond?
Quality	How good does it need to be?
Risk	What happens if it's wrong?
Maintainability	Can the team debug and update it?
Time to ship	How fast can v1 reach production?

For agents, add cost explosion risk, trust boundaries, blast radius, and debuggability. Blast radius is the one to put on the wall. A wrong classification is one bad label. A wrong agent action can delete a database, send an email, or spend \$47K.

Don't ship the demo. Ship the system.

Step 5: Signals

Three types, each doing a different job.

Type	What it tells you	Example
Success	Working looks like this	Resolution rate above 75 percent
Stop	Abandon if this fires	Cost per query above \$0.50
Leading	Week-1 warning sign	Classification confidence below threshold

The success number carries a guardrail, a second metric that must not get worse. The stop number names the fallback you switch to. A stop criterion without a named fallback is just a threat.

Inner loop or outer loop? Inner-loop signals tell you the model is performing. Outer-loop signals tell you the system is solving the right problem. Most teams measure inner only and are surprised six months in.

Write the pre-mortem. *It's six months from now. The project failed. Why?* The mode of thinking that imagines failure is different from the mode that imagines success. You need both.

Pre-commit signals with stakeholders before you build. The pivot conversation later is much easier when "the signal we agreed on has fired" replaces "I think we should stop."

Common mistakes

Mistake	Fix
Vague metric	Ask "If this improves 10 percent, does the business actually care?"
No metric owner	Find the person whose bonus depends on this
Wrong atomic unit	Ask two teammates to define it. If they disagree, you have a problem.
Anchoring on agent	Generate at least one rule-based or pipeline alternative
No failure signals	Define them before you build, not after launch
Inner-loop only	Add at least one outer-loop signal that ties to the business
"We need more data"	Often means "we need to reframe"
"We need a bigger model"	Often means the Oracle Test wasn't run

The key insight

The metric is the frame. When you define the goal, you're defining what success looks like. Atomic unit, alternatives, trade-offs, signals all flow from that.

Get the metric wrong, and you'll optimize the wrong thing perfectly.

GOATS is not something you do once. It's how you think about every AI project. Run it before you build. Run it again when something breaks.